

# LC 934 - Shortest Bridge

McKinsey

Goldman Sachs

Flipkart

## Problem Logic & Notes

**Find min distance between 2 islands** (There are exactly 2 islands in a grid)

*Can we pick any cell from L1 and start BFS?*

**NO**, the nearest cell on L1 from L2 can be somewhere else.

Hence:

**(S1)** as soon as you see a land, start a dfs to push all it's elements in a queue

**(S2)** start a multisource BFS

Island  $\Rightarrow$  water  $\Rightarrow$  water  $\Rightarrow$  Island  
[ ans = 2 ]

**T.C:  $O(n^2) + O(n^2) \rightarrow O(2n^2)$**

BFS

DFS

## C++ Solution

---

```

class Solution {
public:
    vector<pair<int, int>> dir = {{-1, 0}, {0, -1}, {1, 0}, {0, 1}};
    queue<pair<int, int>> q;
    int n;
    int m;

    void dfs(int r, int c, vector<vector<int>>& grid) {

        grid[r][c] = 0;
        q.push({r, c});

        for (auto it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;

            if (nr >= 0 && nc >= 0 && nr < n && nc < m) {
                if (grid[nr][nc] == 1) {
                    dfs(nr, nc, grid);
                }
            }
        }
    }

    int shortestBridge(vector<vector<int>>& grid) {
        n = grid.size();
        m = grid[0].size();
        bool found = false; // 1 hi island pe dfs krna he
        for (int i = 0; i < n && !found; i++) {
            for (int j = 0; j < m && !found; j++) {
                if (grid[i][j] == 1) {
                    found = true;
                    dfs(i, j, grid);
                }
            }
        }

        int ans = 0;
        while (!q.empty()) {
            int currsz = q.size();
            while (currsz--) {
                int r = q.front().first;
                int c = q.front().second;

                q.pop();
                for (auto it : dir) {
                    int nr = r + it.first;
                    int nc = c + it.second;

                    if (nr >= 0 && nc >= 0 && nr < n && nc < m) {

                        if (grid[nr][nc] == 1) {
                            return ans;
                        }
                        if (grid[nr][nc] == 0) {
                            grid[nr][nc] = -1;
                            q.push({nr, nc});
                        }
                    }
                }
            }
            ans++;
        }
    }
};

```

```
        }  
    }  
    ans++;  
}  
return -1;  
}  
};
```